

Debugging di base

Debugging di base

Come trovare e correggere gli errori nei programmi Python.

Perché ti serve fare debugging

Scrivere codice senza errori al primo colpo è praticamente impossibile — anche per i programmatori più esperti al mondo. Il **debugging** (o "debug") è l'arte di trovare e correggere gli errori nel tuo programma.

Pensa a un detective: quando qualcosa non funziona, devi raccogliere indizi, fare ipotesi e testare le tue teorie finché non trovi il colpevole.

I tre tipi di errori

1. Errori di sintassi

Sono come errori grammaticali: hai scritto qualcosa che Python non capisce. Vengono segnalati **prima ancora che il programma parta**.

```
# Manca i due punti alla fine della riga "if"
if x > 0
    print(x)
# SyntaxError: expected ':'
```

Sono i più semplici da correggere: Python ti dice esattamente dove si trova il problema.

2. Errori durante l'esecuzione

Il codice è scritto correttamente, ma qualcosa va storto **mentre il programma sta girando**.

```
lista = [1, 2, 3]
print(lista[10]) # Stai chiedendo il decimo elemento, ma ne esistono solo
3!
# IndexError: list index out of range
```

È come chiedere alla cassiera di un negozio il prodotto numero 10, ma sullo scaffale ce ne sono solo 3.

3. Errori logici

Questi sono i più insidiosi: il programma **gira senza errori**, ma produce risultati sbagliati. Python non se ne accorge — sei tu che devi trovarli.

```
def calcola_media(numeri):
    return sum(numeri) / len(numeri) - 1 # Bug! Il "-1" non dovrebbe
    esserci

print(calcola_media([8, 7, 9])) # Stampa 7.0 invece di 8.0 — sbagliato!
```

Leggere il messaggio di errore (Traceback)

Quando Python trova un errore, stampa un **traceback** — una specie di "mappa" che ti mostra cosa stava succedendo nel momento dell'errore.

```
Traceback (most recent call last):
  File "script.py", line 10, in <module>
    risultato = dividi(10, 0)
  File "script.py", line 4, in dividi
    return a / b
ZeroDivisionError: division by zero
```

Come leggerlo: parti dall'ultima riga e vai su.

- **Ultima riga:** il tipo di errore e la descrizione (`ZeroDivisionError: division by zero`)
- **Righe precedenti:** la sequenza di passaggi che ha portato all'errore (prima è stato chiamato `dividi`, che è a riga 10, e dentro `dividi` l'errore è a riga 4)

Tecnica 1: usare `print()` come spia

Il modo più semplice per capire cosa sta succedendo dentro il tuo programma è inserire delle `print()` temporanee per "spiare" i valori delle variabili.

```
def calcola_media(voti):  
    print(f"DEBUG: voti ricevuti → {voti}")    # Spia: cosa arriva alla  
    funzione?  
    totale = sum(voti)  
    print(f"DEBUG: totale calcolato → {totale}")    # Spia: il totale è  
    giusto?  
    media = totale / len(voti)  
    print(f"DEBUG: media calcolata → {media}")    # Spia: la media è  
    giusta?  
    return media  
  
risultato = calcola_media([8, 7, 9])  
print(f"Risultato finale: {risultato}")
```

Quando hai trovato il bug e corretto il codice, **ricordati di eliminare le righe di debug** prima di consegnare o condividere il programma.

Tecnica 2: il modulo `logging`

Le `print()` funzionano, ma hanno un problema: devi ricordarti di eliminarle tutte. Il modulo `logging` è più elegante: puoi **accenderlo o spegnerlo** con una sola riga.

```
# Questa riga "accende" tutti i messaggi di debug
logging.basicConfig(level=logging.DEBUG)

def calcola_media(voti):
    logging.debug(f"Voti ricevuti: {voti}")
    totale = sum(voti)
    logging.debug(f"Totale: {totale}")
    return totale / len(voti)

calcola_media([8, 7, 9])
```

I livelli di gravità, dal meno al più grave:

Livello	Quando usarlo
DEBUG	Dettagli per il debug
INFO	Informazioni generali ("il programma è partito")
WARNING	Qualcosa di strano, ma il programma continua
ERROR	Qualcosa è andato storto
CRITICAL	Errore gravissimo

Tecnica 3: il debugger interattivo **pdb**

Immagina di poter **mettere in pausa** il tuo programma nel mezzo dell'esecuzione e guardare dentro. È esattamente quello che fa **pdb**.

```
def funzione_problematica(x, y):
    breakpoint() # Il programma si ferma qui e aspetta i tuoi comandi
    risultato = x / y
    return risultato

funzione_problematica(10, 2)
```

Quando il programma si ferma, puoi scrivere comandi:

Comando	Significato
n	Esegui la riga successiva (next)
s	Entra dentro una funzione (step)
p nome_variabile	Stampa il valore di una variabile
c	Continua fino al prossimo punto di pausa (continue)
q	Esci dal debugger (quit)

:::note `breakpoint()` è disponibile da Python 3.7 in poi. Nelle versioni precedenti si usava `import pdb; pdb.set_trace()` .:::

Tecnica 4: il debugger grafico (VS Code o PyCharm)

Se usi un editor come **VS Code** o **PyCharm**, hai a disposizione un debugger grafico molto più comodo di `pdb` :

- Clicca sul numero di riga per aggiungere un **punto di pausa** (breakpoint)
 - Avanza riga per riga con i pulsanti
 - Guarda tutte le variabili in una finestra apposita, senza dover digitare nulla
-

Trucchi utili

Controlla il tipo di una variabile

A volte il bug è che una variabile contiene un tipo di dato inaspettato (ad esempio una stringa invece di un numero).

```
x = qualche_funzione()
print(type(x), x)    # Stampa sia il tipo che il valore
```

Usa **assert** per aggiungere controlli

assert ti permette di scrivere "mi aspetto che questa condizione sia vera — se non lo è, fermati e dimmi perché".

```
def calcola_area(base, altezza):  
    assert base > 0, f"La base deve essere positiva, ma ho ricevuto:  
{base}"  
    assert altezza > 0, f"L'altezza deve essere positiva, ma ho ricevuto:  
{altezza}"  
    return base * altezza  
  
calcola_area(5, 3)      # Funziona, area = 15  
calcola_area(-1, 3)     # AssertionError: La base deve essere positiva, ma  
                        ho ricevuto: -1
```

Isola il problema

Se il bug è in un programma lungo, crea un piccolo programma separato che riproduce solo il problema. Questo ti aiuta a:

- Eliminare tutto il codice non rilevante
- Concentrarti solo sul pezzo problematico
- Testare soluzioni più rapidamente